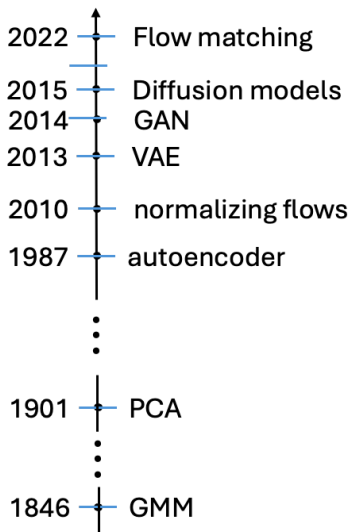


COMM114: Generative AI - PCA and Autoencoder

Guoqiang Zhang

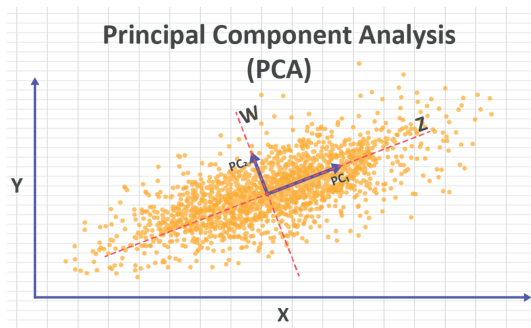
September 2025

History of Generative AI



Principle Component Analysis (PCA)

PCA is a linear dimensionality reduction technique that projects high dimensional data into a low dimensional subspace such that the variance of the projected data is maximized, or equivalently, most information is preserved.



Motivation

- **Dimensionality Reduction:** High-dimensional data can be difficult to analyze. PCA helps simplify the data by identifying the most relevant dimensions (principal components) that capture the majority of the variance.
- **Data Compression:** By reducing dimensions, PCA can lead to more compact data representations, which can be useful for storage and transmission
- **Noise Reduction:** PCA can help filter out noise in the data by focusing on the principal components that represent the true underlying structure, as these tend to capture the most significant patterns and variations.
- **Improved Visualization:** With PCA, it is much easier to visualize low dimensional data.

Basic Idea of PCA

- Input data set $\{\mathbf{x}_i \in \mathbb{R}^d | i = 1, 2, \dots, n\}$
- Let $\boldsymbol{\mu}$ be the mean of the data samples, i.e., $\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$.
- Objective: find a k -dimensional subspace $\mathcal{S} \subset \mathbb{R}^d$ such that $\mathbf{x}_i - \boldsymbol{\mu}$ is well-approximated by its projection onto $\mathcal{S}$.

Definition

The projection of a vector $\mathbf{y}$ onto a subspace $\mathcal{S}$ is defined to be

$$\begin{aligned}\mathbf{y}^* &= \text{Proj}_{\mathcal{S}}(\mathbf{y}) \\ &= \arg \min_{\mathbf{z} \in \mathcal{S}} \|\mathbf{z} - \mathbf{y}\|^2\end{aligned}$$

Remark: The projection $\mathbf{y}^*$ is the best approximation of $\mathbf{y}$ from the perspective of MSE criterion.

Prior Knowledge of Projection

- Let $\{\mathbf{u}_m\}_{m=1}^k$ be an orthonormal basis of the subspace $\mathcal{S}$.
 - Properties: $\mathbf{u}_i^T \mathbf{u}_j = 0$ for all $i \neq j$ and $\|\mathbf{u}_i\| = 1$ for all $i = 1, \dots, k$.
- The projection $\text{Proj}_{\mathcal{S}}(\mathbf{y})$ can be alternatively presented as

$$\begin{aligned}\text{Proj}_{\mathcal{S}}(\mathbf{y}) &= \sum_{m=1}^k z_m \mathbf{u}_m \quad \text{where} \quad z_m = \mathbf{u}_m^T \mathbf{y} \\ &= \mathbf{U} \mathbf{z} \\ &= \mathbf{U} \mathbf{U}^T \mathbf{y}\end{aligned}$$

where $\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_k]$

Solution of PCA

- Input data set $\{\mathbf{x}_i \in \mathbb{R}^d | i = 1, 2, \dots, n\}$
- Let $\boldsymbol{\mu}$ be the mean of the data samples, i.e., $\boldsymbol{\mu} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i$.
- Objective: find a k -dimensional subspace $\mathcal{S} \subset \mathbb{R}^d$ in terms of **the orthonormal matrix** $\mathbf{U}$ such that $\mathbf{x}_i - \boldsymbol{\mu}$ is well-approximated by its projection $\text{Proj}_{\mathcal{S}}(\mathbf{x}_i - \boldsymbol{\mu})$.

Solution

- First define $\boldsymbol{\Sigma} = \frac{1}{n}(\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T$
- Let λ_i and $\mathbf{u}_i$ be the i th eigenvalue and its associated eigenvector of $\boldsymbol{\Sigma}$, where $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0$.
- The orthonormal matrix $\mathbf{U}$ for the PCA subspace $\mathcal{S}$ is given by $\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_k]$.
- The projection $\text{Proj}_{\mathcal{S}}(\mathbf{x}_i - \boldsymbol{\mu})$ for PCA is given by

$$\text{Proj}_{\mathcal{S}}(\mathbf{x}_i - \boldsymbol{\mu}) = \mathbf{U}\mathbf{U}^T(\mathbf{x}_i - \boldsymbol{\mu})$$

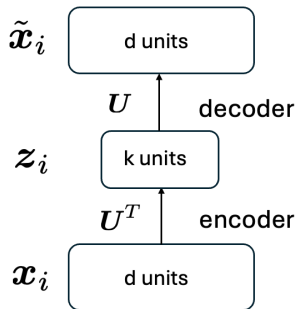
Encoder-Decoder Perspective of PCA

- The **linear encoder** U maps each $\mathbf{x}_i - \boldsymbol{\mu}$ to the k dimensional vector $\mathbf{z}_i$, thus doing dimensionality reduction, where

$$\mathbf{z}_i = U^T(\mathbf{x}_i - \boldsymbol{\mu}).$$

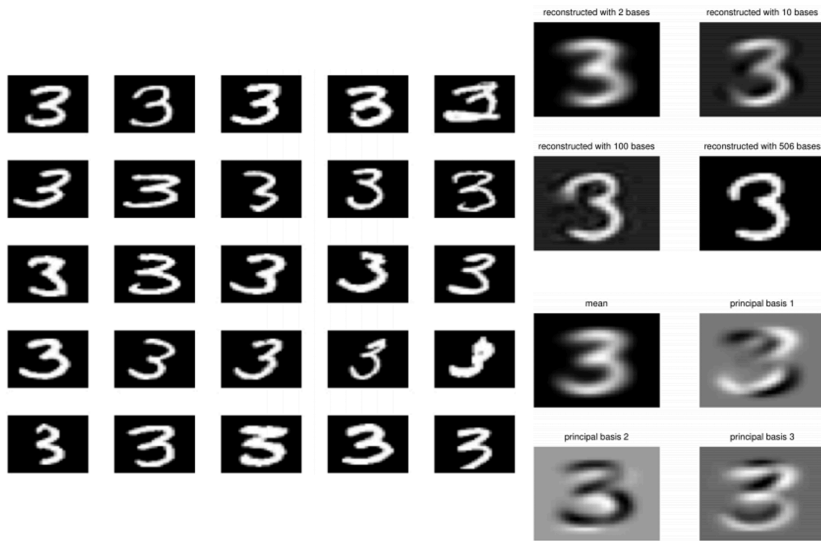
- $\mathbf{z}_i$ is alternatively referred to as the code of the input sample $\mathbf{x}_i - \boldsymbol{\mu}$.
- The **linear decoder** U produces an approximate $\tilde{\mathbf{x}}_i$ of the original sample $\mathbf{x}_i$ based on $\mathbf{z}_i$ via

$$\tilde{\mathbf{x}}_i = U\mathbf{z}_i.$$



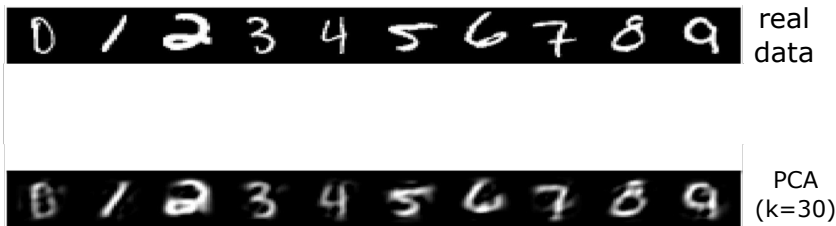
Remark: d units in the figure stands for d components of the vector.

PCA for handwritten digits



picture from Lecture notes by Mengye Ren and Matthew MacKay

PCA for handwritten digits (cont.)

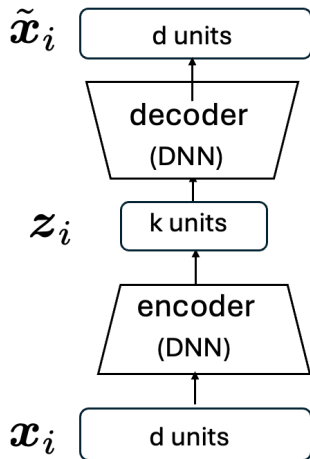


- When k is relatively small, the reconstruction quality of PCA is poor.

picture from Lecture notes by Mengye Ren and Matthew MacKay

Autoencoder as Extension of PCA

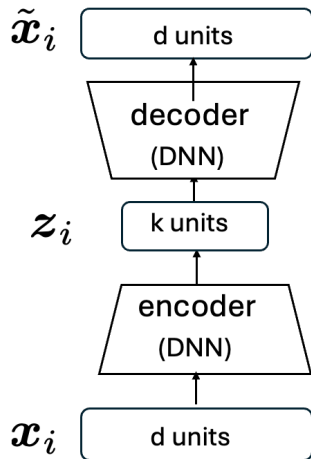
- To improve the reconstruction quality of PCA, autoencoder is designed by replacing the linear encoder and linear decoder of PCA with nonlinear encoder and nonlinear deocder, respectively.
- In principle, the nonlinerity of the encoder and deocder provides more flexibility and degree of freedom when minimizing the reconstruction error between the input and output.
- As a result, autoencoders learn to project the data, not onto a subspace, but onto a nonlinear manifold.



Components of Autoencoder

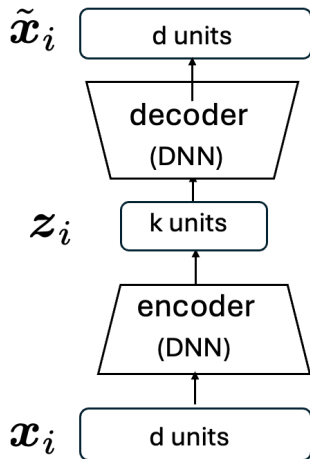
An autoencoder consists of three components:

- Encoder: it compresses the input via a neural network and produces the code.
- Code
- Decoder: it reconstruct the input via another neural network by using only the code.



Definition of Autoencoder

- Autoencoders are a specific type of feedforward neural networks where the output is a prediction of the input. They compress the input into a lower-dimensional code and then reconstruct the output from this representation. The code is a “compression” of the input, also called the latent-space representation.



Why Autoencoder?

- Compression (reducing size of input data)
- Learn abstract features in an unsupervised manner
 - Unlabeled data can be much more rich than labeled data.
 - Similar to PCA, with autoencoder, it is easier to visualize low dimensional data.

Training of Autoencoder by MSE

1. Input data set $\{\mathbf{x}_i \in \mathbb{R}^d | i = 1, 2, \dots, n\}$
2. Build proper DNN architectures for encoder and decoder.
3. Train the encoder and decoder by

$$\min \underbrace{\frac{1}{n} \sum_{i=1}^n \|\mathbf{x}_i - \text{decoder}(\text{encoder}(\mathbf{x}_i))\|^2}_{\text{MSE}}$$

Remark: MSE refers to "mean squared error".

Training of Autoencoder by binary cross entropy (BCE)

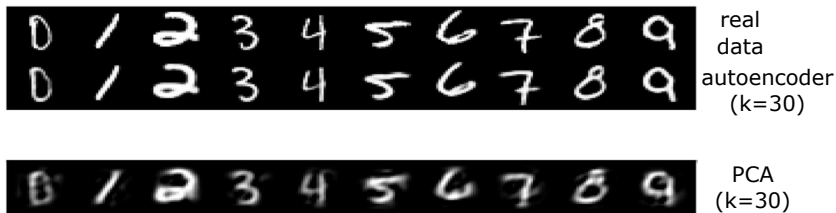
1. Input data set $\{\mathbf{x}_i \in \mathbb{R}^d | i = 1, 2, \dots, n\}$
 - Assumption: each component $\mathbf{x}_i[j] \in [0, 1]$
 - We treat $\mathbf{x}_i[j]$ as a probability.
2. Build proper DNN architectures for encoder and decoder.
3. Train the encoder and decoder by

$$\min \underbrace{\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^d \left[-\mathbf{x}_i[j] \log(\tilde{\mathbf{x}}_i[j]) - (1 - \mathbf{x}_i[j]) \log(1 - \tilde{\mathbf{x}}_i[j]) \right]}_{\text{BCE}}$$

where $\tilde{\mathbf{x}}_i = \text{decoder}(\text{encoder}(\mathbf{x}_i))$

Remark: MSE refers to "mean squared error".

Autoencoder for handwritten digits



- Autoencoder produces much better reconstruction quality than PCA with the same size of the code.

References

- Lecture notes by Roger Grosse
- Lecture notes by Mengye Ren and Matthew MacKay

Questions?